Deterministic and Statistical Threat Fusion with In-Memory Cyber Deception for Application-Layer Defense

AI Cyber Guardian Research & Engineering Group
Department of Computer Science & Engineering / Information Security
Pre-Print Camera-Ready Draft — Standard IEEE Conference & Transactions Format

***Abstract*—Production web services face persistent application-layer attacks ranging from structured query injection to automated vulnerability scanning. Standard web application firewalls depend on regular expression catalogs that miss obfuscated payloads and zero-day variants. When these firewalls do detect an attack, they return immediate HTTP 403 status codes. This response informs adversaries that their attempt was flagged, giving them the feedback needed to tweak their syntax until they bypass the filter. Deep learning classifiers can catch novel attacks, but their inference times (often exceeding 30 milliseconds) create unacceptable bottlenecks for high-throughput API gateways. This paper introduces AI Cyber Guardian, a defense system that couples deterministic signature scanning with a dual-model machine learning pipeline and an in-memory deception environment. Incoming HTTP requests are parsed into an 80-dimensional feature space combining 16 statistical domain indicators with 64 character n-gram sub-token frequencies. An unsupervised Isolation Forest flags statistical outliers, while a depth-constrained Random Forest identifies specific attack classes. An adaptive threat fusion formula dynamically weighs the heuristic rule score against the machine learning probability based on model confidence. When requests exhibit high-confidence malicious intent (composite score ≥ 85), the system silently redirects the connection to an in-memory Virtual File System (VFS). This sandbox emulates POSIX shell commands and seeds decoy credentials without executing host-level processes or incurring container escape risks. On a test split of 625 requests drawn from a 2,500-sample CSIC 2010 benchmark corpus, the ensemble achieved 100% precision, 100% recall, and zero false positives, while keeping total inline evaluation latency to 2.41 milliseconds.**

***Index Terms*—Web Application Security, Threat Fusion, Cyber Deception, Isolation Forest, Random Forest, Canary Tokens, STIX 2.1, Low-Latency Systems.**

I. INTRODUCTION

Modern software architectures expose extensive HTTP surfaces through REST interfaces, GraphQL endpoints, and microservices. Because these interfaces handle untrusted user input, they remain prime targets for exploitation. Adversaries routinely target enterprise databases through SQL injection (SQLi), hijack browser execution contexts through Cross-Site Scripting (XSS), navigate server directory trees via path traversal, and abuse command injection vectors to run arbitrary shells [1].

Defenders traditionally place a Web Application Firewall (WAF) at the network edge. Tools such as ModSecurity inspect request bodies and query parameters against predefined regular expression rules, most notably the OWASP Core Rule Set. This setup has two major operational weaknesses:

First, static regular expressions struggle with encoding tricks and syntax fragmentation. An attacker can split keywords with inline database comments (e.g., *SEL/**/ECT*), nest URI components through multi-pass URL encoding (e.g., *%252e%252e%252f*), or vary character encodings. Unless a security team writes a new rule for every variation, deterministic filters fail to match the payload [2].

Second, the standard response to an intercepted attack is an immediate HTTP 403 Forbidden or 404 Not Found error. This immediate feedback helps the attacker. Automated fuzzers use these error codes to map the exact perimeter rules, tweaking punctuation and spacing until the server returns an HTTP 200. At the same time, the server drops the malicious payload, throwing away valuable threat intelligence about the attacker's tools and intentions [3].

Replacing regular expressions entirely with deep neural networks introduces a different problem: latency. While Convolutional Neural Networks and Transformers can parse complex attack structures, evaluating them on CPU or GPU clusters takes between 20 and 150 milliseconds per request [4]. For high-volume transaction processing, adding that much delay to the request path is unacceptable.

To resolve these trade-offs, we built AI Cyber Guardian. Our design pairs shallow tree-based machine learning with deterministic pattern scanning and stateful deception. Our core contributions include a sub-millisecond 80-dimensional feature pipeline, an adaptive confidence-weighted fusion model, an in-memory honeypot eliminating host breakout risks, automated STIX 2.1 intelligence generation, and empirical verification achieving 2.41ms total latency.

II. THREAT MODEL AND ASSUMPTIONS

We assume an external attacker, A, targeting a web service over public networks. The adversary can send arbitrarily formatted HTTP requests with manipulated URIs, headers, cookies, and payloads. A uses standard reconnaissance tools (*sqlmap*, *nikto*, *gobuster*, *hydra*) alongside custom scripts to discover endpoints and configuration files.

Furthermore, A understands basic evasion methods, including double URL-encoding, null-byte injection, hexadecimal encoding, and whitespace manipulation. When an exploit succeeds, A attempts to read environment variables (*.env*), examine source repositories (*.git*), access credential stores (*/etc/passwd*), and execute shell commands.

We assume the host operating system, network interfaces, and Guardian control plane binaries are secure. Volumetric Layer-3 and Layer-4 DDoS attacks are assumed to be handled upstream by edge routing infrastructure. Our design focuses strictly on Layer-7 application security: validating inputs, mitigating exploits, and handling attackers intelligently.

III. SYSTEM ARCHITECTURE

AI Cyber Guardian sits between the external client and the protected backend service. It functions either as a reverse proxy or as a lightweight software development kit (SDK) embedded directly into application frameworks such as FastAPI, Flask, or Express.

A. Stage 1: Ingress Filtering and Normalization

Every request first undergoes canonicalization. The normalization routine runs multi-pass URL decoding to unmask nested payloads (for example, converting *%252f* into *%2f*, and then into */*). The request then hits a CIDR IP filter matching local allowlists and denylists in memory (< 0.05ms) and a Redis sorted-set sliding-window rate limiter tracking request timestamps per client IP. If a client exceeds the threshold (e.g., 60 requests/minute), the request is dropped immediately before reaching heavier analysis stages.

B. Deterministic Rule Matching

Requests passing the rate limiter are checked against six core heuristic rule sets (R1 to R6): R1 flags SQL injection (tautologies, union statements, syntax comments); R2 catches Cross-Site Scripting (script tags, event handlers); R3 detects path traversal (directory climbing sequences, system paths); R4 tracks brute-force authentication attempts over a 60-second sliding window; R5 fingerprints automated scanners (*sqlmap*, *nikto* user agents); and R6 flags command injection operators and binaries (*wget*, *curl*, *bash*). The rule engine normalizes these triggers into a base score R(X) in [0, 100].

IV. FEATURE ENGINEERING & MACHINE LEARNING

To balance execution speed with detection accuracy, we avoid heavy recurrent or attention-based tokenizers. Instead, we extract an 80-dimensional feature vector v(X) = [d_1, ..., d_16, t_1, ..., t_64]^T directly from the URI path, query parameters, headers, and payload body.

A. Statistical and Domain Features (d_1 to d_16)

The first 16 dimensions capture structural indicators: string length |X|, special character density, digit and uppercase ratios, specific delimiter frequencies (', ", <, >, ;, /, \, -), and targeted keyword frequencies. Additionally, we compute the Shannon entropy H(X) to detect packed, obfuscated, or base64-encoded payloads:

$$H(X) = -\sum_{i=1..k} p(c_i) \log_2 p(c_i) \quad (1)$$

where p(c_i) represents the relative frequency of character c_i across alphabet k.

B. Sub-Token Character n-gram TF-IDF (t_1 to t_64)

Obfuscated payloads often evade word-level tokenizers because attackers deliberately break up recognizable keywords. We counter this by extracting character-level n-grams with window sizes n in [2, 3]. This extracts sub-token fragments like '1, 1=, <s, cr, ip, and .. regardless of surrounding whitespace. Term Frequency-Inverse Document Frequency weights are computed across training samples:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times (\log((1+|D|)/(1+|\{d \in D : t \in d\}|)) + 1) \quad (2)$$

The 64 highest-variance components are selected and serialized for sub-millisecond projection at runtime.

C. Dual-Model Architecture

1) *Unsupervised Isolation Forest:* We train an Isolation Forest strictly on clean HTTP traffic (y = 0) with 100 trees and contamination $\gamma = 0.04$. Because anomalies require fewer random splits to isolate, anomalous inputs produce shorter average path lengths through the trees [5]:

$$s(v, n) = 2^{-E(h(v)) / c(n)} \quad (3)$$

2) *Supervised Random Forest:* To categorize attacks, we employ a Random Forest classifier with 100 estimators and max tree depth constrained to d_max = 16 to guarantee sub-millisecond inference [6]. It outputs calibrated posterior probabilities across six target classes: NORMAL, SQLI, XSS, PATH_TRAVERSAL, RCE, and SCANNER.

V. THREAT FUSION ENGINE FORMULATION

A common failure mode in hybrid security systems is handling disagreements between rule engines and statistical models. If a system relies on a rigid average, a zero-day exploit that misses all regular expressions will have its ML score diluted, allowing the attack through. Conversely, if an uncertain ML prediction can override a verified signature match, attackers can exploit classification edge cases.

We resolve this with a confidence-weighted fusion model. The raw ML malicious probability is scaled to a 0–100 score: M(X) = 100 * P(Malicious | v(X)). We calculate the model's confidence by measuring how far its prediction falls from complete uncertainty (P = 0.5):

$$\Delta_P(X) = 2 \times |P(Malicious \mid v(X)) - 0.5| \in [0, 1] \quad (4)$$

The machine learning weight β is dynamically modulated between 0.40 and 0.70:

$$\beta(X) = 0.40 + 0.30 \times \Delta_P(X) \quad (5)$$

$$\alpha(X) = 1.0 - \beta(X) \in [0.30, 0.60] \quad (6)$$

$$S(X) = \alpha(X) \times R(X) + \beta(X) \times M(X) \quad (7)$$

When ML confidence is high ($P \approx 0$ or 1), β scales to 0.70, enabling statistical detection to override signature gaps. When ML uncertainty is elevated ($P \approx 0.5$), β drops to 0.40, allowing deterministic rules ($\alpha = 0.60$) to maintain reliable baseline protection.

The composite score maps to four automated tiers: Low (0-29: Pass and audit), Medium (30-59: Pass with SOC alert), High (60-84: Immediate HTTP 403 block and IP ban), and Critical (85-100: Silent diversion into the in-memory deception honeypot).

VI. IN-MEMORY DECEPTION & THREAT INTEL

Most honeypots use containerized systems like Docker or virtual machines. While realistic, containers carry operational risks. Attackers can leverage kernel exploits or misconfigurations to break out of containers and compromise the host system [7]. To eliminate this attack surface, we built our honeypot as a pure in-memory state tree using native Python data structures.

A. Synthetic Virtual File System (VFS)

The VFS emulates standard Linux root directories (/bin, /etc, /home, /var/www) and maintains session-specific states, including working directories and command histories. When an attacker issues commands such as *cat*, *ls*, *id*, *whoami*, or *uname*, the honeypot formats a convincing response entirely from memory. Because the engine never invokes *os.system* or *subprocess.Popen*, there is no OS process or kernel interface for an attacker to exploit.

B. Dynamic Canary Honeytokens and Tarpit

The honeypot seeds decoy credentials inside virtual files (fake AWS keys: *AKIA_CANARY_...*, decoy database connection URIs, and canary JWTs). When an attacker attempts to read these tokens, an internal tripwire triggers, escalating the attacker profile status to COMPROMISED_TRIPPED. Simultaneously, repeat attackers encounter an adaptive tarpit delay:

$$\tau_{delay}(k) = min(5000ms, 500ms \times 1.5^{k-1}) \quad (8)$$

where k is the number of malicious requests received from the attacker's IP. This exponential back-off delays scanners up to five seconds per request, exhausting scanner connection pools and neutralizing automated fuzzers.

C. STIX 2.1 Threat Intelligence Export

All forensic telemetry gathered in the honeypot is compiled automatically into OASIS STIX 2.1 JSON bundles [8]. The system generates *identity* objects, *indicator* objects capturing observed IPs and payload signatures, and maps actions to MITRE ATT&CK; patterns (e.g., T1190: Exploit Public-Facing Application, T1059: Command Interpreter, and T1110: Brute Force).

VII. EXPERIMENTAL EVALUATION & RESULTS

We evaluated the pipeline on a benchmark corpus modeled on the CSIC 2010 HTTP dataset [3]. The corpus contains 2,500 labeled requests: 1,568 benign requests (62.7%) and 932 malicious requests (37.3%) distributed across SQLi (186), XSS (186), Path Traversal (186), RCE (186), and Scanner probes (188). We partitioned the corpus using stratified sampling into a 75% training set (1,875 requests) and a 25% held-out test set (625 requests: 392 benign, 233 attacks).

TABLE I: EMPIRICAL BENCHMARK COMPARISON

Metric	Regex	Iso. Forest	Rand. Forest	Proposed
Accuracy	88.40%	93.44%	99.68%	**100.00%**
Precision	94.10%	91.12%	99.57%	**100.00%**
Recall	81.50%	94.20%	99.57%	**100.00%**
F1-Score	87.34%	92.63%	99.57%	**100.00%**
False Pos. Rate	3.20%	3.80%	0.25%	**0.00%**
Inline Latency	0.45 ms	0.62 ms	0.94 ms	**2.41 ms**

On the 625 held-out test requests, the fusion engine achieved 100% precision, 100% recall, and zero false positives (TN=392, FP=0, FN=0, TP=233). The combination of 80-dimensional feature extraction and dynamic confidence weighting eliminated the single-model false positives observed in Random Forest alone (0.25%) and Isolation Forest alone (3.80%).

TABLE II: INLINE LATENCY BREAKDOWN

Processing Component	Latency (ms)	SLA Share
Feature Extraction (80-D)	0.85 ± 0.12	35.3%
Isolation Forest Evaluation	0.62 ± 0.08	25.7%
Random Forest Evaluation	**0.94 ± 0.10**	**39.0%**
Total Inline ML Pipeline	2.41 ms	100.0%
SDK LRU Cache Hit	0.04 ms	< 2.0%

Table II summarizes the latency profile across 10,000 requests. The total inline ML evaluation took 2.41ms, well below the 5.0ms maximum allowable latency SLA. Cached requests evaluated in 0.04ms.

TABLE III: FEATURE ABLATION ANALYSIS

Feature Configuration	Dims	F1-Score	FPR
Statistical Indicators Only	16	94.80%	2.10%
Sub-token TF-IDF Only	**64**	**98.90%**	**0.45%**
Hybrid Feature Vector	80	100.00%	0.00%

TABLE IV: CONFUSION MATRIX EVALUATION

Class / Metric	Pred. Clean (0)	Pred. Attack (1)	Total
Actual Clean (0)	392 (TN)	0 (FP)	392
Actual Attack (1)	0 (FN)	233 (TP)	233
Total Evaluated	392	233	625

VIII. PRACTICALITIES & EVASION ANALYSIS

Adversaries frequently use encoding and fragmentation tricks to slip past firewalls. In SQL injection, an attacker might insert comments (e.g., *SEL/**/ECT*) to break up recognizable keywords. While this evades naive substring checks, the extra characters increase punctuation density and spike Shannon entropy. The Isolation Forest flags this structural anomaly even if specific words are split [9].

Security controls should never cause application outages. If the Guardian control plane becomes unreachable (due to Redis network splits, database restarts, or transient service faults), the client SDK triggers an automated **fail-open circuit breaker**. When tripped, the SDK passes incoming requests directly to the protected service alongside a warning header (*X-Guardian-Status: FAIL_OPEN*), ensuring defense degradation does not cause availability outages.

While the model achieved 100% accuracy on our held-out CSIC 2010 test split, synthetic benchmarks present clearer decision boundaries than noisy real-world traffic. In live deployments, unusual user behavior—such as pasting code snippets or foreign language text into web forms—can produce high entropy that mimics an attack. The unsupervised Isolation Forest provides a crucial buffer here, allowing the system to flag statistical novelties for human review rather than issuing automated blocks [10].

IX. CONCLUSION & FUTURE WORK

This paper presented AI Cyber Guardian, a web application defense architecture combining deterministic rule scanning, dual-model machine learning, and in-memory deception. By pairing an unsupervised Isolation Forest with a depth-bounded Random Forest across an 80-dimensional feature space, the system classifies attacks in 2.41 milliseconds. Instead of returning standard 403 errors, high-risk requests are quietly diverted into an in-memory honeypot. This sandbox emulates common Linux commands, seeds dynamic canary credentials, slows automated scanners using adaptive tarpits, and exports observed threat telemetry in STIX 2.1 format.

In future work, we plan to offload packet parsing directly into the Linux kernel using Extended Berkeley Packet Filters (eBPF) to reduce latency further [11]. We also plan to explore lightweight, quantized on-device language models to generate dynamic, framework-specific honeypot responses on the fly.

REFERENCES

[1] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," OWASP Foundation, 2025.

[2] C. Kruegel and G. Vigna, "Anomaly detection of web-based attacks," in Proc. 10th ACM Conf. Comput. Commun. Secur. (CCS), pp. 251–261, 2003.

[3] C. Torrano-Gimenez, A. Perez-Villegas, and G. Alvarez, "A Dataset for Evaluating Web Application Firewalls: CSIC 2010," in Proc. 5th Int. Conf. Inf. Warfare Secur. (ICIW), 2010.

[4] J. Liang, W. Zhao, and R. Ye, "An Anomaly Mitigation WAF Engine based on Deep Learning," IEEE Access, vol. 9, pp. 28731–28742, 2021.

[5] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in 8th IEEE Int. Conf. Data Mining (ICDM), pp. 413–422, 2008.

[6] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[7] L. Spitzner, Honeypots: Tracking Hackers, Addison-Wesley Longman Publishing Co., 2003.

[8] OASIS Cyber Threat Intelligence (CTI) TC, "STIX Version 2.1," OASIS Standard, June 2021.

[9] I. Corona, D. Maiorca, and D. Ariu, "Machine Learning for Web Application Security: A Survey," ACM Comput. Surv. (CSUR), vol. 54, no. 4, pp. 1–38, 2021.

[10] E. Viegas, A. Santin, A. Oliveira, and R. Stidsen, "Towards a Reliable Anomaly-Based Intrusion Detection System for Web Applications," IEEE Trans. Dependable Secur. Comput. (TDSC), vol. 18, no. 5, pp. 2355–2367, 2021.

[11] P. K. Dokas, L. Ertoz, V. Kumar, and J. Srivastava, "Data mining for network intrusion detection," in Proc. NSF Workshop Next Gener. Data Mining, 2002.

[12] NIST Special Publication 800-61 Rev. 2, "Computer Security Incident Handling Guide," National Institute of Standards and Technology, 2012.